

DBSCAN & Pipelines

Krishna Sekhar
Padma Venkatraman

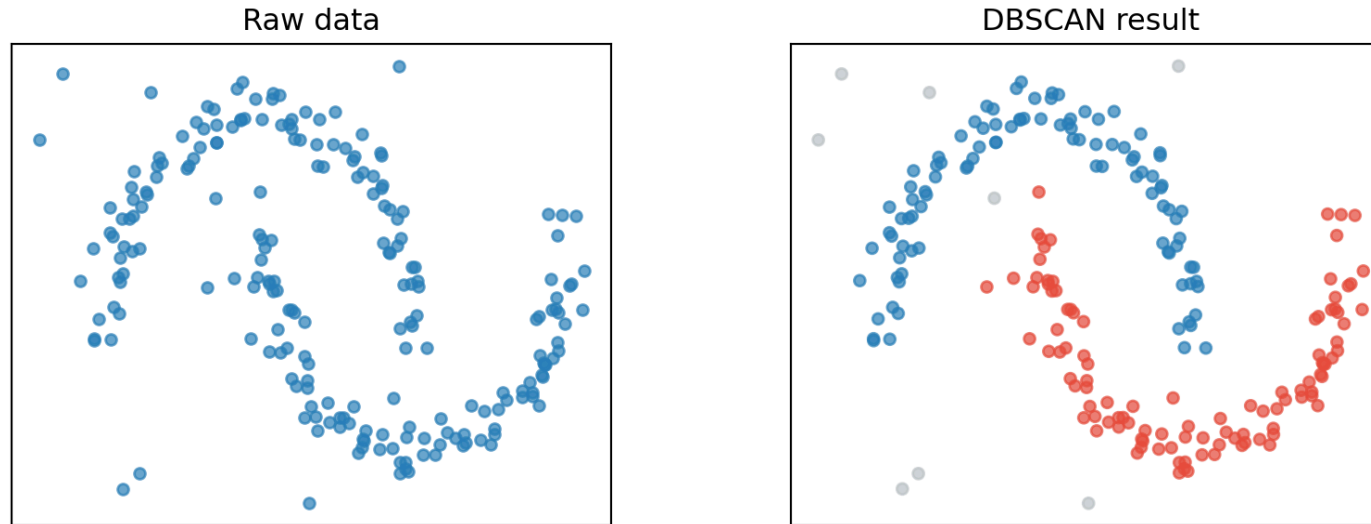
Recap

- **K-means**: fast, simple, but needs K and assumes spherical clusters
- Fails on non-convex shapes, unequal sizes, and outliers
- We saw that DBSCAN handles the concentric rings K-means couldn't

Now: how does DBSCAN actually work?

DBSCAN — the intuition

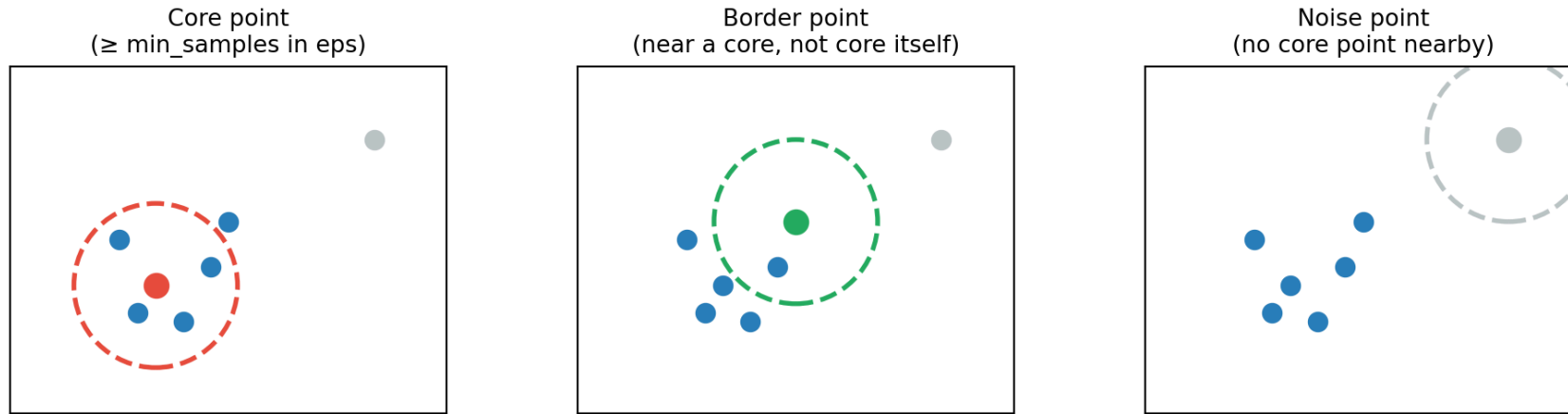
Clusters are **dense regions** separated by sparse gaps.



DBSCAN finds clusters by walking through dense regions. Isolated points (grey) are labelled as *noise* — no cluster needed for everything (unlike K-Means)

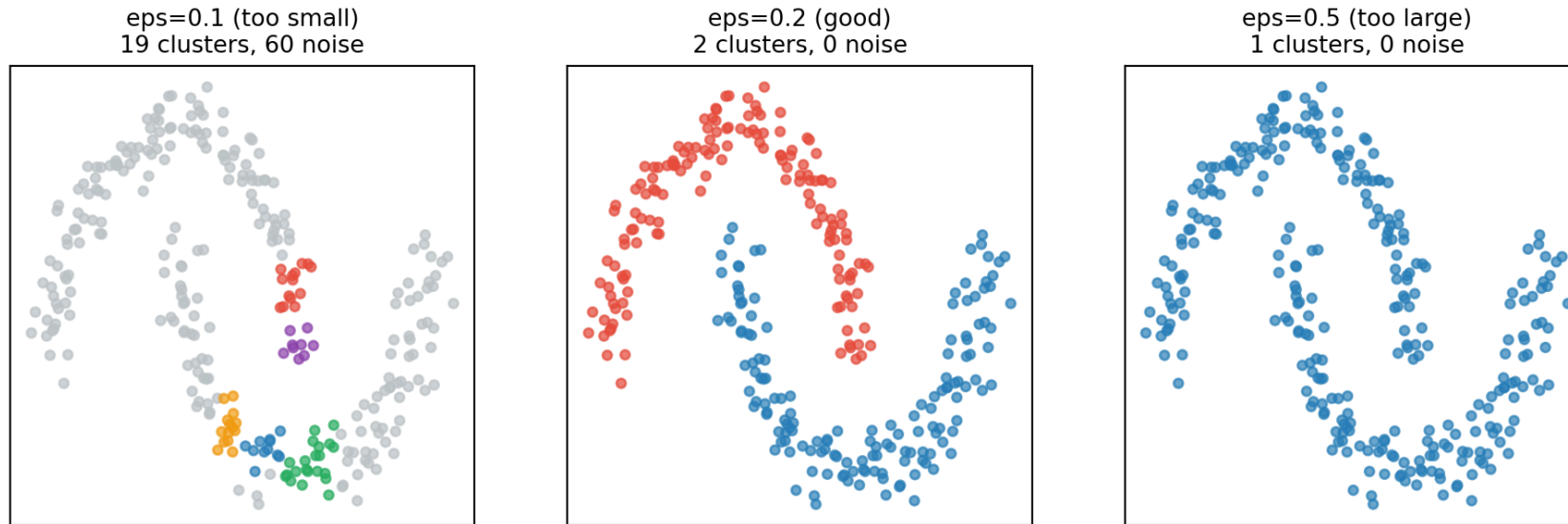
DBSCAN — how it works

Two parameters: *eps* (neighbourhood radius) and *min_samples* (density threshold).



Clusters form by connecting core points whose neighbourhoods overlap. Border points join the nearest cluster. Noise stays unassigned.

Tuning eps and min_samples

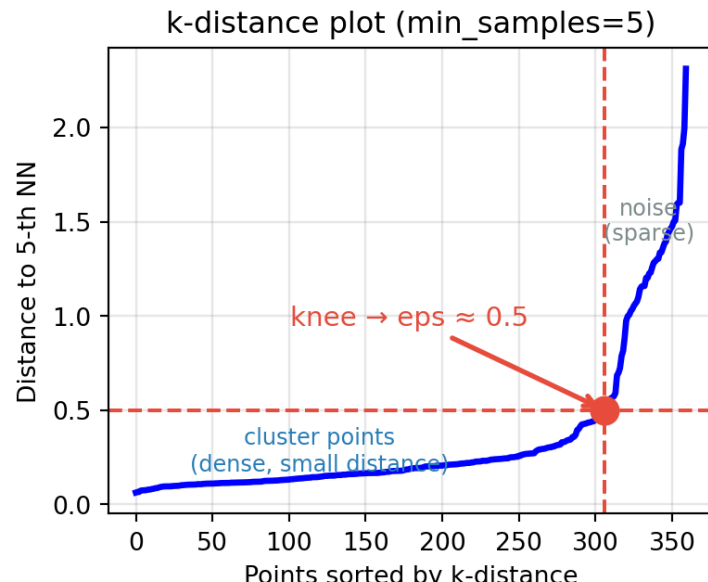


Too small → everything is noise. Too large → distinct clusters merge. `min_samples` controls how conservative the density threshold is.

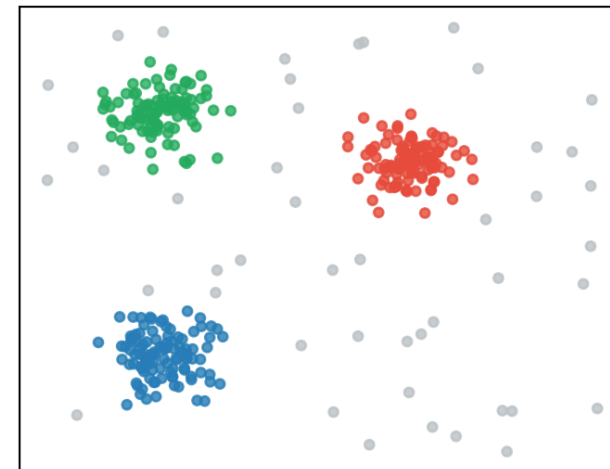
A heuristic: the k-distance plot

The *k-distance plot* turns “tune *eps* by eye” into something systematic.

- Pick *min_samples*, then for each point measure the distance to its *min_samples*-th nearest neighbour; sort ascending
- Cluster points sit on a **flat plateau** (dense $\rightarrow$ small distance); noise points form the **steep tail** (sparse $\rightarrow$ large distance)
- The **knee** between them $\approx$ a good *eps* — the density threshold separating clusters from noise



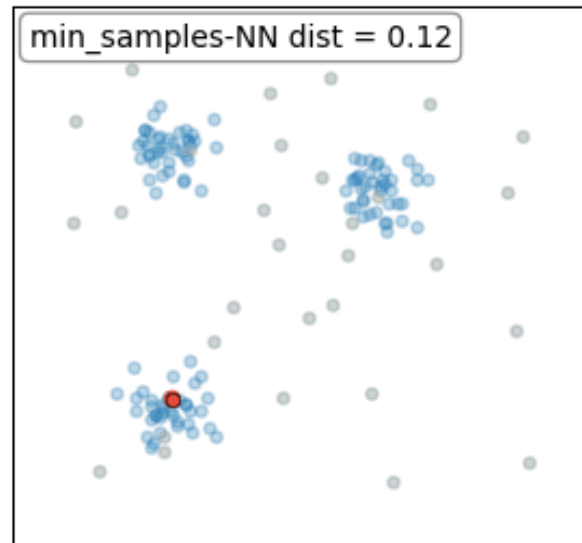
Read eps off the knee $\rightarrow$ 3 clusters, noise in grey



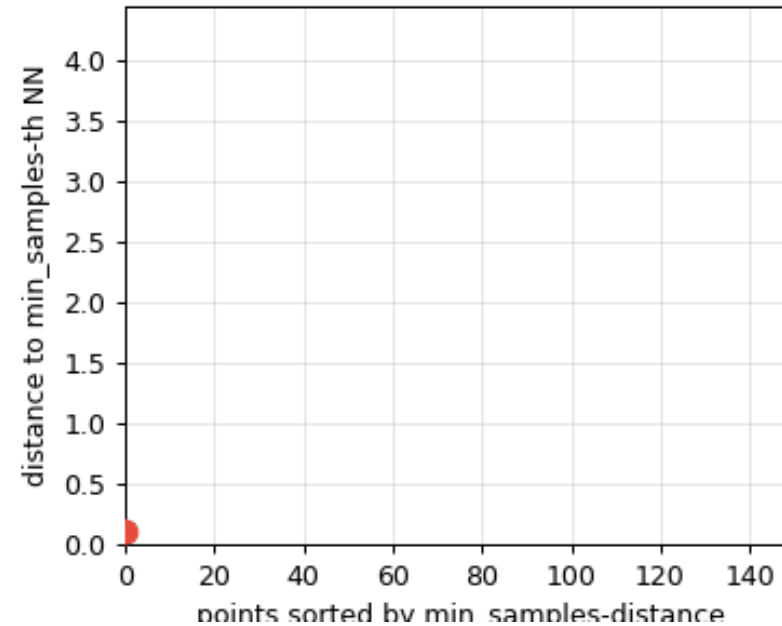
Why the k-distance plot works

A point's distance to its `min_samples`-th neighbour is the *smallest eps at which it becomes a core point*. Measure that for every point, sort, and watch the curve build:

radius to the min_samples-th neighbour (here 5)

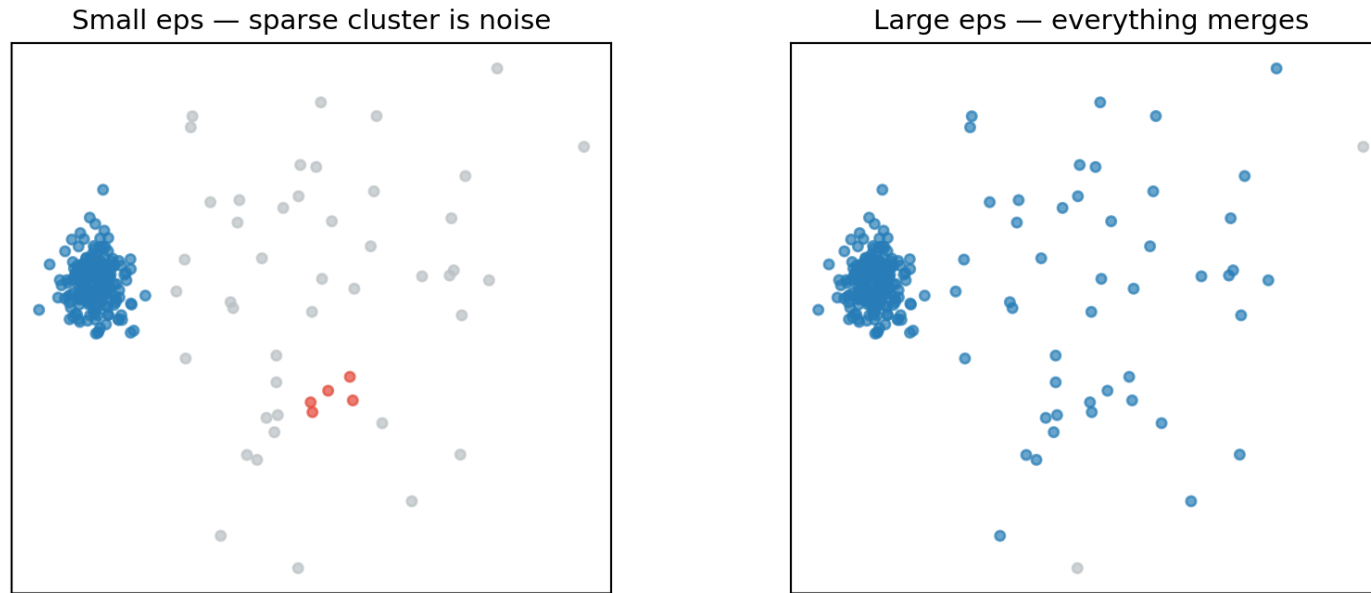


the curve fills in, one point at a time



Dense points need a tiny radius (flat plateau); sparse points need a big one (steep tail). The **knee** is where that jump happens.

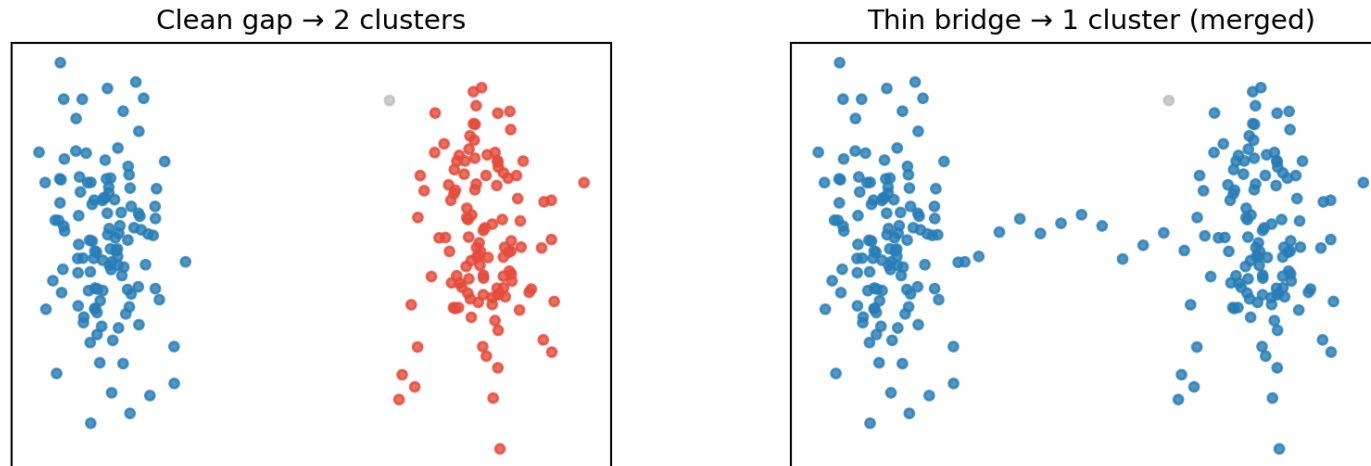
Failure mode 1: variable density



One `eps` can't fit both a tight and a diffuse cluster. Small `eps` → sparse cluster becomes noise. Large `eps` → both clusters merge.

Failure mode 2: chaining

DBSCAN grows clusters by *density-reachability* — walking point to point through *eps*-neighbourhoods. A thin **bridge** of points chains two distinct clusters into one.



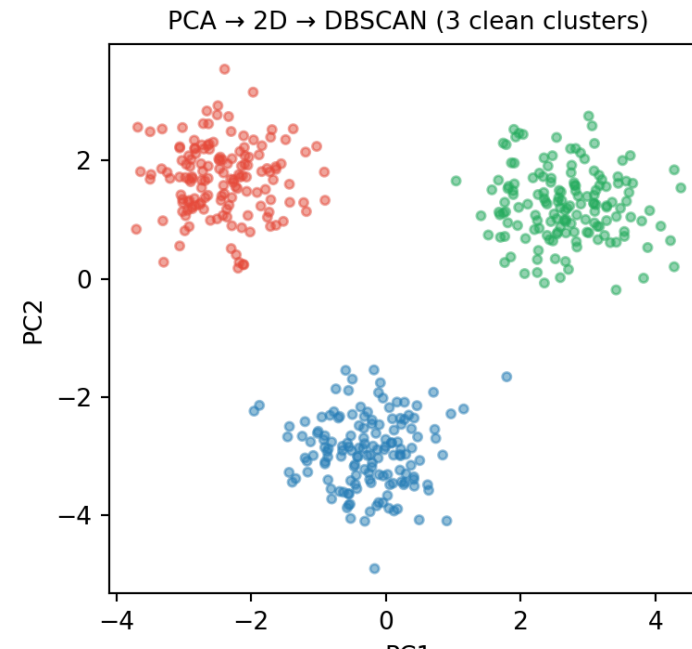
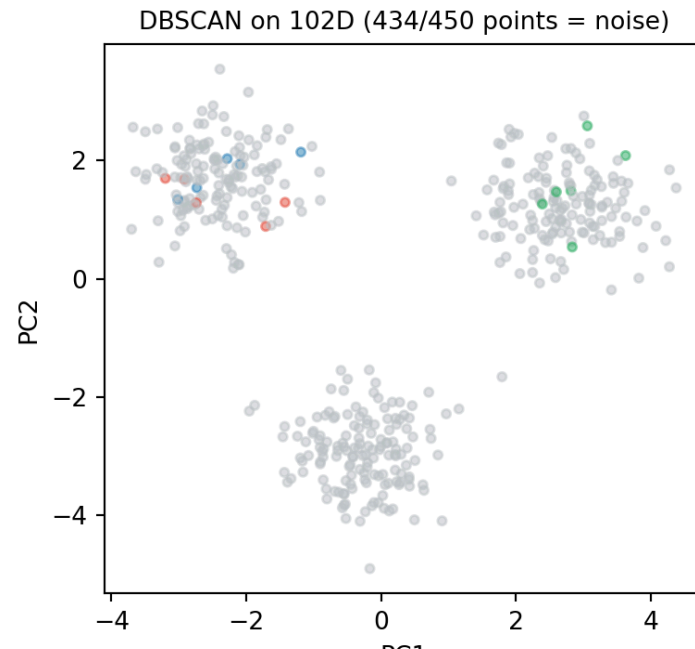
The same *eps* that separates the clusters merges them once a sparse bridge connects them.

Putting It Together: Pipelines

PCA + clustering

In high dimensions, distances **concentrate** — the L1/L2 metrics from Day 1 stop discriminating, and every pair of points looks roughly equally far apart. Clustering algorithms that rely on distances (like DBSCAN) struggle.

A common pipeline: **reduce first, then cluster.**



Choosing a method

There's no universal best algorithm. Pick based on what you know:

You know...	Try...
Number of clusters (K)	K-means (fast, simple)
Clusters are elliptical	GMM
Arbitrary shapes, outliers present	DBSCAN
High-dimensional data	PCA first, then cluster
Nothing at all	Start with K-means + elbow, visualise, iterate

Rules of thumb:

- K-means is a good default — fast, easy to interpret, works well when clusters are roughly round and similar-sized
- Always visualise your results. If the clustering looks wrong, it probably is.
- Try more than one method — if they disagree, that tells you something about your data

Summary

- **DBSCAN:** density-based, no K , handles arbitrary shapes and noise
- **Tuning:** `eps` and `min_samples` control what counts as dense
- **Failure mode:** can't handle clusters with very different densities
- **Pipelines:** PCA + clustering strips noise, improves results
- **Method choice:** depends on what you know about your data